MATH 547 Week 1 Reading Packet

Source excerpts

Source excerpt	Packet pp.
Shalev-Shwartz and Ben-David, *Understanding Machine Learning*	
Chapter 2, Sections 2.1–2.3; printed pp. 13–20.	
Learning model, empirical risk minimization, realizability, and the finite-class argument.	2–9
Shalev-Shwartz and Ben-David, *Understanding Machine Learning*	
Chapter 3, Sections 3.1–3.2; printed pp. 22–27.	
PAC and agnostic PAC learning, data-generating distributions, and general loss functions.	10–15
Koltchinskii, *Oracle Inequalities in Empirical Risk Minimization and Sparse Recovery Problems*	
Chapter 1, Sections 1.1–1.2 (opening); printed pp. 1–3.	
Risk minimization, empirical risk minimization, and excess risk.	16–18
Shalev-Shwartz and Ben-David, *Understanding Machine Learning*	
Chapter 4, Sections 4.1–4.2; printed pp. 31–34.	
Representative samples, uniform convergence, ERM transfer, and finite-class bounds.	19–22

All printed page numbers refer to the pagination in the original sources. Packet page numbers count this guide as page 1.

2 Day 1

A Gentle Start

Let us begin our mathematical analysis by showing how successful learning can be achieved in a relatively simplified setting. Imagine you have just arrived in some small Pacific island. You soon find out that papayas are a significant ingredient in the local diet. However, you have never before tasted papayas. You have to learn how to predict whether a papaya you see in the market is tasty or not. First, you need to decide which features of a papaya your prediction should be based on. On the basis of your previous experience with other fruits, you decide to use two features: the papaya's color, ranging from dark green, through orange and red to dark brown, and the papaya's softness, ranging from rock hard to mushy. Your input for figuring out your prediction rule is a sample of papayas that you have examined for color and softness and then tasted and found out whether they were tasty or not. Let us analyze this task as a demonstration of the considerations involved in learning problems.

Our first step is to describe a formal model aimed to capture such learning tasks.

2.1 A FORMAL MODEL – THE STATISTICAL LEARNING FRAMEWORK

The learner's input: In the basic statistical learning setting, the learner has access to the following:

- **Domain set:** An arbitrary set, $\mathcal{X}$. This is the set of objects that we may wish to label. For example, in the papaya learning problem mentioned before, the domain set will be the set of all papayas. Usually, these domain points will be represented by a vector of *features* (like the papaya's color and softness). We also refer to domain points as *instances* and to $\mathcal{X}$ as instance space.
- **Label set:** For our current discussion, we will restrict the label set to be a two-element set, usually $\{0, 1\}$ or $\{-1, +1\}$. Let $\mathcal{Y}$ denote our set of possible labels. For our papayas example, let $\mathcal{Y}$ be $\{0, 1\}$, where 1 represents being tasty and 0 stands for being not-tasty.
- **Training data:** $S = ((x_1, y_1) \ldots (x_m, y_m))$ is a finite sequence of pairs in $\mathcal{X} \times \mathcal{Y}$: that is, a sequence of labeled domain points. This is the input that the

learner has access to (like a set of papayas that have been tasted and their color, softness, and tastiness). Such labeled examples are often called *training examples*. We sometimes also refer to S as a *training set*.[1]

The learner's output: The learner is requested to output a *prediction rule*, $h : \mathcal{X} \to \mathcal{Y}$. This function is also called a *predictor*, a *hypothesis*, or a *classifier*. The predictor can be used to predict the label of new domain points. In our papayas example, it is a rule that our learner will employ to predict whether future papayas he examines in the farmers' market are going to be tasty or not. We use the notation $A(S)$ to denote the hypothesis that a learning algorithm, A, returns upon receiving the training sequence S.

A simple data-generation model We now explain how the training data is generated. First, we assume that the instances (the papayas we encounter) are generated by some probability distribution (in this case, representing the environment). Let us denote that probability distribution over $\mathcal{X}$ by $\mathcal{D}$. It is important to note that we do not assume that the learner knows anything about this distribution. For the type of learning tasks we discuss, this could be any arbitrary probability distribution. As to the labels, in the current discussion we assume that there is some "correct" labeling function, $f : \mathcal{X} \to \mathcal{Y}$, and that $y_i = f(x_i)$ for all i. This assumption will be relaxed in the next chapter. The labeling function is unknown to the learner. In fact, this is just what the learner is trying to figure out. In summary, each pair in the training data S is generated by first sampling a point x_i according to $\mathcal{D}$ and then labeling it by f.

Measures of success: We define the *error of a classifier* to be the probability that it does not predict the correct label on a random data point generated by the aforementioned underlying distribution. That is, the error of h is the probability to draw a random instance x, according to the distribution $\mathcal{D}$, such that $h(x)$ does not equal $f(x)$.

Formally, given a domain subset,[2] $A \subset \mathcal{X}$, the probability distribution, $\mathcal{D}$, assigns a number, $\mathcal{D}(A)$, which determines how likely it is to observe a point $x \in A$. In many cases, we refer to A as an event and express it using a function $\pi : \mathcal{X} \to \{0, 1\}$, namely, $A = \{x \in \mathcal{X} : \pi(x) = 1\}$. In that case, we also use the notation $\mathbb{P}_{x \sim \mathcal{D}}[\pi(x)]$ to express $\mathcal{D}(A)$.

We define the error of a prediction rule, $h : \mathcal{X} \to \mathcal{Y}$, to be

$$L_{\mathcal{D},f}(h) \stackrel{\text{def}}{=} \mathbb{P}_{x \sim \mathcal{D}}[h(x) \neq f(x)] \stackrel{\text{def}}{=} \mathcal{D}(\{x : h(x) \neq f(x)\}). \tag{2.1}$$

That is, the error of such h is the probability of randomly choosing an example x for which $h(x) \neq f(x)$. The subscript $(\mathcal{D}, f)$ indicates that the error is measured with respect to the probability distribution $\mathcal{D}$ and the correct labeling function f. We omit this subscript when it is clear from the context. $L_{(\mathcal{D},f)}(h)$ has several synonymous names such as the *generalization error*, the *risk*, or the *true error* of h, and we will use these names interchangeably throughout

[1] Despite the "set" notation, S is a sequence. In particular, the same example may appear twice in S and some algorithms can take into account the order of examples in S.

[2] Strictly speaking, we should be more careful and require that A is a member of some σ-algebra of subsets of $\mathcal{X}$, over which $\mathcal{D}$ is defined. We will formally define our measurability assumptions in the next chapter.

the book. We use the letter L for the error, since we view this error as the *loss* of the learner. We will later also discuss other possible formulations of such loss.

A note about the information available to the learner The learner is blind to the underlying distribution $\mathcal{D}$ over the world and to the labeling function f. In our papayas example, we have just arrived in a new island and we have no clue as to how papayas are distributed and how to predict their tastiness. The only way the learner can interact with the environment is through observing the training set.

In the next section we describe a simple learning paradigm for the preceding setup and analyze its performance.

2.2 EMPIRICAL RISK MINIMIZATION

As mentioned earlier, a learning algorithm receives as input a training set S, sampled from an unknown distribution $\mathcal{D}$ and labeled by some target function f, and should output a predictor $h_S : \mathcal{X} \to \mathcal{Y}$ (the subscript S emphasizes the fact that the output predictor depends on S). The goal of the algorithm is to find h_S that minimizes the error with respect to the unknown $\mathcal{D}$ and f.

Since the learner does not know what $\mathcal{D}$ and f are, the true error is not directly available to the learner. A useful notion of error that can be calculated by the learner is the *training error* – the error the classifier incurs over the training sample:

$$L_S(h) \stackrel{\text{def}}{=} \frac{|\{i \in [m] : h(x_i) \neq y_i\}|}{m}, \tag{2.2}$$

where $[m] = \{1, \ldots, m\}$.

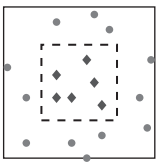
The terms *empirical error* and *empirical risk* are often used interchangeably for this error.

Since the training sample is the snapshot of the world that is available to the learner, it makes sense to search for a solution that works well on that data. This learning paradigm – coming up with a predictor h that minimizes $L_S(h)$ – is called *Empirical Risk Minimization* or ERM for short.

2.2.1 Something May Go Wrong – Overfitting

Although the ERM rule seems very natural, without being careful, this approach may fail miserably.

To demonstrate such a failure, let us go back to the problem of learning to predict the taste of a papaya on the basis of its softness and color. Consider a sample as depicted in the following:



Assume that the probability distribution $\mathcal{D}$ is such that instances are distributed uniformly within the gray square and the labeling function, f, determines the label to be 1 if the instance is within the inner square, and 0 otherwise. The area of the gray square in the picture is 2 and the area of the inner square is 1. Consider the following predictor:

$$h_S(x) = \begin{cases} y_i & \text{if } \exists i \in [m] \text{ s.t. } x_i = x \\ 0 & \text{otherwise.} \end{cases} \tag{2.3}$$

While this predictor might seem rather artificial, in Exercise 2.1 we show a natural representation of it using polynomials. Clearly, no matter what the sample is, $L_S(h_S) = 0$, and therefore this predictor may be chosen by an ERM algorithm (it is one of the empirical-minimum-cost hypotheses; no classifier can have smaller error). On the other hand, the true error of any classifier that predicts the label 1 only on a finite number of instances is, in this case, $1/2$. Thus, $L_{\mathcal{D}}(h_S) = 1/2$. We have found a predictor whose performance on the training set is excellent, yet its performance on the true "world" is very poor. This phenomenon is called *overfitting*. Intuitively, overfitting occurs when our hypothesis fits the training data "too well" (perhaps like the everyday experience that a person who provides a perfect detailed explanation for each of his single actions may raise suspicion).

2.3 EMPIRICAL RISK MINIMIZATION WITH INDUCTIVE BIAS

We have just demonstrated that the ERM rule might lead to overfitting. Rather than giving up on the ERM paradigm, we will look for ways to rectify it. We will search for conditions under which there is a guarantee that ERM does not overfit, namely, conditions under which when the ERM predictor has good performance with respect to the training data, it is also highly likely to perform well over the underlying data distribution.

A common solution is to apply the ERM learning rule over a restricted search space. Formally, the learner should choose in advance (before seeing the data) a set of predictors. This set is called a *hypothesis class* and is denoted by $\mathcal{H}$. Each $h \in \mathcal{H}$ is a function mapping from $\mathcal{X}$ to $\mathcal{Y}$. For a given class $\mathcal{H}$, and a training sample, S, the $\text{ERM}_{\mathcal{H}}$ learner uses the ERM rule to choose a predictor $h \in \mathcal{H}$, with the lowest possible error over S. Formally,

$$\text{ERM}_{\mathcal{H}}(S) \in \operatorname*{argmin}_{h \in \mathcal{H}} L_S(h),$$

where argmin stands for the set of hypotheses in $\mathcal{H}$ that achieve the minimum value of $L_S(h)$ over $\mathcal{H}$. By restricting the learner to choosing a predictor from $\mathcal{H}$, we *bias* it toward a particular set of predictors. Such restrictions are often called an *inductive bias*. Since the choice of such a restriction is determined before the learner sees the training data, it should ideally be based on some prior knowledge about the problem to be learned. For example, for the papaya taste prediction problem we may choose the class $\mathcal{H}$ to be the set of predictors that are determined by axis aligned rectangles (in the space determined by the color and softness coordinates). We will later show that $\text{ERM}_{\mathcal{H}}$ over this class is guaranteed not to overfit. On the other hand, the example of overfitting that we have seen previously, demonstrates that choosing $\mathcal{H}$

to be a class of predictors that includes all functions that assign the value 1 to a finite set of domain points does not suffice to guarantee that $\text{ERM}_{\mathcal{H}}$ will not overfit.

A fundamental question in learning theory is, over which hypothesis classes $\text{ERM}_{\mathcal{H}}$ learning will not result in overfitting. We will study this question later in the book.

Intuitively, choosing a more restricted hypothesis class better protects us against overfitting but at the same time might cause us a stronger inductive bias. We will get back to this fundamental tradeoff later.

2.3.1 Finite Hypothesis Classes

The simplest type of restriction on a class is imposing an upper bound on its size (that is, the number of predictors h in $\mathcal{H}$). In this section, we show that if $\mathcal{H}$ is a finite class then $\text{ERM}_{\mathcal{H}}$ will not overfit, provided it is based on a sufficiently large training sample (this size requirement will depend on the size of $\mathcal{H}$).

Limiting the learner to prediction rules within some finite hypothesis class may be considered as a reasonably mild restriction. For example, $\mathcal{H}$ can be the set of all predictors that can be implemented by a C++ program written in at most 10^9 bits of code. In our papayas example, we mentioned previously the class of axis aligned rectangles. While this is an infinite class, if we discretize the representation of real numbers, say, by using a 64 bits floating-point representation, the hypothesis class becomes a finite class.

Let us now analyze the performance of the $\text{ERM}_{\mathcal{H}}$ learning rule assuming that $\mathcal{H}$ is a finite class. For a training sample, S, labeled according to some $f : \mathcal{X} \to \mathcal{Y}$, let h_S denote a result of applying ERM_H to S, namely,

$$h_S \in \operatorname*{argmin}_{h \in \mathcal{H}} L_S(h). \tag{2.4}$$

In this chapter, we make the following simplifying assumption (which will be relaxed in the next chapter).

Definition 2.1 (The Realizability Assumption). There exists $h^\star \in \mathcal{H}$ s.t. $L_{(\mathcal{D},f)}(h^\star) = 0$. Note that this assumption implies that with probability 1 over random samples, S, where the instances of S are sampled according to $\mathcal{D}$ and are labeled by f, we have $L_S(h^\star) = 0$.

The realizability assumption implies that for every ERM hypothesis we have that[3] $L_S(h_S) = 0$. However, we are interested in the *true* risk of h_S, $L_{(\mathcal{D},f)}(h_S)$, rather than its empirical risk.

Clearly, any guarantee on the error with respect to the underlying distribution, $\mathcal{D}$, for an algorithm that has access only to a sample S should depend on the relationship between $\mathcal{D}$ and S. The common assumption in statistical machine learning is that the training sample S is generated by sampling points from the distribution $\mathcal{D}$ independently of each other. Formally,

[3] Mathematically speaking, this holds with probability 1. To simplify the presentation, we sometimes omit the "with probability 1" specifier.

The i.i.d. assumption: The examples in the training set are independently and identically distributed (i.i.d.) according to the distribution $\mathcal{D}$. That is, every x_i in S is freshly sampled according to $\mathcal{D}$ and then labeled according to the labeling function, f. We denote this assumption by $S \sim \mathcal{D}^m$ where m is the size of S, and $\mathcal{D}^m$ denotes the probability over m-tuples induced by applying $\mathcal{D}$ to pick each element of the tuple independently of the other members of the tuple.

Intuitively, the training set S is a window through which the learner gets partial information about the distribution $\mathcal{D}$ over the world and the labeling function, f. The larger the sample gets, the more likely it is to reflect more accurately the distribution and labeling used to generate it.

Since $L_{(\mathcal{D},f)}(h_S)$ depends on the training set, S, and that training set is picked by a random process, there is randomness in the choice of the predictor h_S and, consequently, in the risk $L_{(\mathcal{D},f)}(h_S)$. Formally, we say that it is a random variable. It is not realistic to expect that with full certainty S will suffice to direct the learner toward a good classifier (from the point of view of $\mathcal{D}$), as there is always some probability that the sampled training data happens to be very nonrepresentative of the underlying $\mathcal{D}$. If we go back to the papaya tasting example, there is always some (small) chance that all the papayas we have happened to taste were not tasty, in spite of the fact that, say, 70% of the papayas in our island are tasty. In such a case, $\text{ERM}_{\mathcal{H}}(S)$ may be the constant function that labels every papaya as "not tasty" (and has 70% error on the true distribution of papapyas in the island). We will therefore address the *probability* to sample a training set for which $L_{(\mathcal{D},f)}(h_S)$ is not too large. Usually, we denote the probability of getting a nonrepresentative sample by δ, and call $(1-\delta)$ the *confidence parameter* of our prediction.

On top of that, since we cannot guarantee perfect label prediction, we introduce another parameter for the quality of prediction, the *accuracy parameter*, commonly denoted by ϵ. We interpret the event $L_{(\mathcal{D},f)}(h_S) > \epsilon$ as a failure of the learner, while if $L_{(\mathcal{D},f)}(h_S) \leq \epsilon$ we view the output of the algorithm as an approximately correct predictor. Therefore (fixing some labeling function $f : \mathcal{X} \to \mathcal{Y}$), we are interested in upper bounding the probability to sample m-tuple of instances that will lead to failure of the learner. Formally, let $S|_x = (x_1, \ldots, x_m)$ be the instances of the training set. We would like to upper bound

$$\mathcal{D}^m(\{S|_x : L_{(\mathcal{D},f)}(h_S) > \epsilon\}).$$

Let $\mathcal{H}_B$ be the set of "bad" hypotheses, that is,

$$\mathcal{H}_B = \{h \in \mathcal{H} : L_{(\mathcal{D},f)}(h) > \epsilon\}.$$

In addition, let

$$M = \{S|_x : \exists h \in \mathcal{H}_B, L_S(h) = 0\}$$

be the set of misleading samples: Namely, for every $S|_x \in M$, there is a "bad" hypothesis, $h \in \mathcal{H}_B$, that looks like a "good" hypothesis on $S|_x$. Now, recall that we would like to bound the probability of the event $L_{(\mathcal{D},f)}(h_S) > \epsilon$. But, since the realizability assumption implies that $L_S(h_S) = 0$, it follows that the event $L_{(\mathcal{D},f)}(h_S) > \epsilon$ can only happen if for some $h \in \mathcal{H}_B$ we have $L_S(h) = 0$. In other words, this event will

only happen if our sample is in the set of misleading samples, M. Formally, we have shown that

$$\{S|_x : L_{(\mathcal{D},f)}(h_S) > \epsilon\} \leq M.$$

Note that we can rewrite M as

$$M = \bigcup_{h \in \mathcal{H}_B} \{S|_x : L_S(h) = 0\}. \tag{2.5}$$

Hence,

$$\mathcal{D}^m(\{S|_x : L_{(\mathcal{D},f)}(h_S) > \epsilon\}) \leq \mathcal{D}^m(M) = \mathcal{D}^m(\cup_{h \in \mathcal{H}_B} \{S|_x : L_S(h) = 0\}). \tag{2.6}$$

Next, we upper bound the right-hand side of the preceding equation using the *union bound* – a basic property of probabilities.

Lemma 2.2 (Union Bound)**.** *For any two sets A, B and a distribution $\mathcal{D}$ we have*

$$\mathcal{D}(A \cup B) \leq \mathcal{D}(A) + \mathcal{D}(B).$$

Applying the union bound to the right-hand side of Equation (2.6) yields

$$\mathcal{D}^m(\{S|_x : L_{(\mathcal{D},f)}(h_S) > \epsilon\}) \leq \sum_{h \in \mathcal{H}_B} \mathcal{D}^m(\{S|_x : L_S(h) = 0\}). \tag{2.7}$$

Next, let us bound each summand of the right-hand side of the preceding inequality. Fix some "bad" hypothesis $h \in \mathcal{H}_B$. The event $L_S(h) = 0$ is equivalent to the event $\forall i, h(x_i) = f(x_i)$. Since the examples in the training set are sampled i.i.d. we get that

$$\begin{aligned}\mathcal{D}^m(\{S|_x : L_S(h) = 0\}) &= \mathcal{D}^m(\{S|_x : \forall i, h(x_i) = f(x_i)\}) \\ &= \prod_{i=1}^{m} \mathcal{D}(\{x_i : h(x_i) = f(x_i)\}).\end{aligned} \tag{2.8}$$

For each individual sampling of an element of the training set we have

$$\mathcal{D}(\{x_i : h(x_i) = y_i\}) = 1 - L_{(\mathcal{D},f)}(h) \leq 1 - \epsilon,$$

where the last inequality follows from the fact that $h \in \mathcal{H}_B$. Combining the previous equation with Equation (2.8) and using the inequality $1 - \epsilon \leq e^{-\epsilon}$ we obtain that for every $h \in \mathcal{H}_B$,

$$\mathcal{D}^m(\{S|_x : L_S(h) = 0\}) \leq (1 - \epsilon)^m \leq e^{-\epsilon m}. \tag{2.9}$$

Combining this equation with Equation (2.7) we conclude that

$$\mathcal{D}^m(\{S|_x : L_{(\mathcal{D},f)}(h_S) > \epsilon\}) \leq |\mathcal{H}_B| e^{-\epsilon m} \leq |\mathcal{H}| e^{-\epsilon m}.$$

A graphical illustration which explains how we used the union bound is given in Figure 2.1.

Corollary 2.3. *Let $\mathcal{H}$ be a finite hypothesis class. Let $\delta \in (0,1)$ and $\epsilon > 0$ and let m be an integer that satisfies*

$$m \geq \frac{\log(|\mathcal{H}|/\delta)}{\epsilon}.$$

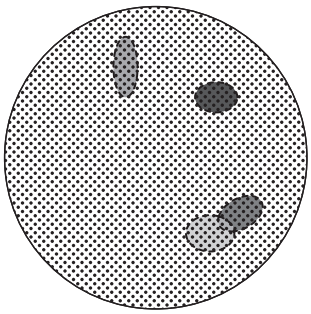


Figure 2.1. Each point in the large circle represents a possible m-tuple of instances. Each colored oval represents the set of "misleading" m-tuple of instances for some "bad" predictor $h \in \mathcal{H}_B$. The ERM can potentially overfit whenever it gets a misleading training set S. That is, for some $h \in \mathcal{H}_B$ we have $L_S(h) = 0$. Equation (2.9) guarantees that for each individual bad hypothesis, $h \in \mathcal{H}_B$, at most $(1-\epsilon)^m$-fraction of the training sets would be misleading. In particular, the larger m is, the smaller each of these colored ovals becomes. The union bound formalizes the fact that the area representing the training sets that are misleading with respect to some $h \in \mathcal{H}_B$ (that is, the training sets in M) is at most the sum of the areas of the colored ovals. Therefore, it is bounded by $|\mathcal{H}_B|$ times the maximum size of a colored oval. Any sample S outside the colored ovals cannot cause the ERM rule to overfit.

Then, for any labeling function, f, and for any distribution, $\mathcal{D}$, for which the realizability assumption holds (that is, for some $h \in \mathcal{H}$, $L_{(\mathcal{D},f)}(h) = 0$), with probability of at least $1-\delta$ over the choice of an i.i.d. sample S of size m, we have that for every ERM hypothesis, h_S, it holds that

$$L_{(\mathcal{D},f)}(h_S) \leq \epsilon.$$

The preceeding corollary tells us that for a sufficiently large m, the $\text{ERM}_{\mathcal{H}}$ rule over a finite hypothesis class will be *probably* (with confidence $1-\delta$) *approximately* (up to an error of ϵ) correct. In the next chapter we formally define the model of Probably Approximately Correct (PAC) learning.

2.4 EXERCISES

2.1 **Overfitting of polynomial matching:** We have shown that the predictor defined in Equation (2.3) leads to overfitting. While this predictor seems to be very unnatural, the goal of this exercise is to show that it can be described as a thresholded polynomial. That is, show that given a training set $S = \{(\mathbf{x}_i, f(\mathbf{x}_i))\}_{i=1}^m \subseteq (\mathbb{R}^d \times \{0,1\})^m$, there exists a polynomial p_S such that $h_S(\mathbf{x}) = 1$ if and only if $p_S(\mathbf{x}) \geq 0$, where h_S is as defined in Equation (2.3). It follows that learning the class of all thresholded polynomials using the ERM rule may lead to overfitting.

2.2 Let $\mathcal{H}$ be a class of binary classifiers over a domain $\mathcal{X}$. Let $\mathcal{D}$ be an unknown distribution over $\mathcal{X}$, and let f be the target hypothesis in $\mathcal{H}$. Fix some $h \in \mathcal{H}$. Show that the expected value of $L_S(h)$ over the choice of $S|_x$ equals $L_{(\mathcal{D},f)}(h)$, namely,

$$\mathop{\mathbb{E}}_{S|_x \sim \mathcal{D}^m}[L_S(h)] = L_{(\mathcal{D},f)}(h).$$

2.3 **Axis aligned rectangles:** An axis aligned rectangle classifier in the plane is a classifier that assigns the value 1 to a point if and only if it is inside a certain rectangle.

3

A Formal Learning Model

In this chapter we define our main formal learning model – the PAC learning model and its extensions. We will consider other notions of learnability in Chapter 7.

3.1 PAC LEARNING

In the previous chapter we have shown that for a finite hypothesis class, if the ERM rule with respect to that class is applied on a sufficiently large training sample (whose size is independent of the underlying distribution or labeling function) then the output hypothesis will be probably approximately correct. More generally, we now define*Probably Approximately Correct* (PAC) learning.

Definition 3.1 (PAC Learnability). A hypothesis class $\mathcal{H}$ is PAC learnable if there exist a function $m_{\mathcal{H}} : (0,1)^2 \to \mathbb{N}$ and a learning algorithm with the following property: For every $\epsilon, \delta \in (0,1)$, for every distribution $\mathcal{D}$ over $\mathcal{X}$, and for every labeling function $f : \mathcal{X} \to \{0,1\}$, if the realizable assumption holds with respect to $\mathcal{H}, \mathcal{D}, f$, then when running the learning algorithm on $m \geq m_{\mathcal{H}}(\epsilon, \delta)$ i.i.d. examples generated by $\mathcal{D}$ and labeled by f, the algorithm returns a hypothesis h such that, with probability of at least $1 - \delta$ (over the choice of the examples), $L_{(\mathcal{D},f)}(h) \leq \epsilon$.

The definition of Probably Approximately Correct learnability contains two approximation parameters. The accuracy parameter ϵ determines how far the output classifier can be from the optimal one (this corresponds to the "approximately correct"), and a confidence parameter δ indicating how likely the classifier is to meet that accuracy requirement (corresponds to the "probably" part of "PAC"). Under the data access model that we are investigating, these approximations are inevitable. Since the training set is randomly generated, there may always be a small chance that it will happen to be noninformative (for example, there is always some chance that the training set will contain only one domain point, sampled over and over again). Furthermore, even when we are lucky enough to get a training sample that does faithfully represent $\mathcal{D}$, because it is just a finite sample, there may always be some fine details of $\mathcal{D}$ that it fails to reflect. Our accuracy parameter, ϵ, allows "forgiving" the learner's classifier for making minor errors.

Sample Complexity

The function $m_{\mathcal{H}} : (0,1)^2 \to \mathbb{N}$ determines the *sample complexity* of learning $\mathcal{H}$: that is, how many examples are required to guarantee a probably approximately correct solution. The sample complexity is a function of the accuracy (ϵ) and confidence (δ) parameters. It also depends on properties of the hypothesis class $\mathcal{H}$ – for example, for a finite class we showed that the sample complexity depends on log the size of $\mathcal{H}$.

Note that if $\mathcal{H}$ is PAC learnable, there are many functions $m_{\mathcal{H}}$ that satisfy the requirements given in the definition of PAC learnability. Therefore, to be precise, we will define the sample complexity of learning $\mathcal{H}$ to be the "minimal function," in the sense that for any ϵ, δ, $m_{\mathcal{H}}(\epsilon, \delta)$ is the minimal integer that satisfies the requirements of PAC learning with accuracy ϵ and confidence δ.

Let us now recall the conclusion of the analysis of finite hypothesis classes from the previous chapter. It can be rephrased as stating:

Corollary 3.2. *Every finite hypothesis class is PAC learnable with sample complexity*

$$m_{\mathcal{H}}(\epsilon, \delta) \leq \left\lceil \frac{\log(|\mathcal{H}|/\delta)}{\epsilon} \right\rceil .$$

There are infinite classes that are learnable as well (see, for example, Exercise 3.3). Later on we will show that what determines the PAC learnability of a class is not its finiteness but rather a combinatorial measure called the *VC dimension*.

3.2 A MORE GENERAL LEARNING MODEL

The model we have just described can be readily generalized, so that it can be made relevant to a wider scope of learning tasks. We consider generalizations in two aspects:

Removing the Realizability Assumption

We have required that the learning algorithm succeeds on a pair of data distribution $\mathcal{D}$ and labeling function f provided that the realizability assumption is met. For practical learning tasks, this assumption may be too strong (can we really guarantee that there is a rectangle in the color-hardness space that *fully determines* which papayas are tasty?). In the next subsection, we will describe the *agnostic PAC* model in which this realizability assumption is waived.

Learning Problems beyond Binary Classification

The learning task that we have been discussing so far has to do with predicting a binary label to a given example (like being tasty or not). However, many learning tasks take a different form. For example, one may wish to predict a real valued number (say, the temperature at 9:00 p.m. tomorrow) or a label picked from a finite set of labels (like the topic of the main story in tomorrow's paper). It turns out that our analysis of learning can be readily extended to such and many other scenarios by allowing a variety of loss functions. We shall discuss that in Section 3.2.2 later.

3.2.1 Releasing the Realizability Assumption – Agnostic PAC Learning

A More Realistic Model for the Data-Generating Distribution

Recall that the realizability assumption requires that there exists $h^\star \in \mathcal{H}$ such that $\mathbb{P}_{x\sim\mathcal{D}}[h^\star(x) = f(x)] = 1$. In many practical problems this assumption does not hold. Furthermore, it is maybe more realistic not to assume that the labels are fully determined by the features we measure on input elements (in the case of the papayas, it is plausible that two papayas of the same color and softness will have different taste). In the following, we relax the realizability assumption by replacing the "target labeling function" with a more flexible notion, a data-labels generating distribution.

Formally, from now on, let $\mathcal{D}$ be a probability distribution over $\mathcal{X} \times \mathcal{Y}$, where, as before, $\mathcal{X}$ is our domain set and $\mathcal{Y}$ is a set of labels (usually we will consider $\mathcal{Y} = \{0, 1\}$). That is, $\mathcal{D}$ is a *joint distribution* over domain points and labels. One can view such a distribution as being composed of two parts: a distribution $\mathcal{D}_x$ over unlabeled domain points (sometimes called the *marginal distribution*) and a *conditional* probability over labels for each domain point, $\mathcal{D}((x, y)|x)$. In the papaya example, $\mathcal{D}_x$ determines the probability of encountering a papaya whose color and hardness fall in some color-hardness values domain, and the conditional probability is the probability that a papaya with color and hardness represented by x is tasty. Indeed, such modeling allows for two papayas that share the same color and hardness to belong to different taste categories.

The empirical and the True Error Revised

For a probability distribution, $\mathcal{D}$, over $\mathcal{X} \times \mathcal{Y}$, one can measure how likely h is to make an error when labeled points are randomly drawn according to $\mathcal{D}$. We redefine the true error (or risk) of a prediction rule h to be

$$L_{\mathcal{D}}(h) \stackrel{\text{def}}{=} \mathop{\mathbb{P}}_{(x,y)\sim\mathcal{D}}[h(x) \neq y] \stackrel{\text{def}}{=} \mathcal{D}(\{(x, y) : h(x) \neq y\}). \tag{3.1}$$

We would like to find a predictor, h, for which that error will be minimized. However, the learner does not know the data generating $\mathcal{D}$. What the learner does have access to is the training data, S. The definition of the empirical risk remains the same as before, namely,

$$L_S(h) \stackrel{\text{def}}{=} \frac{|\{i \in [m] : h(x_i) \neq y_i\}|}{m}.$$

Given S, a learner can compute $L_S(h)$ for any function $h : X \to \{0, 1\}$. Note that $L_S(h) = L_{D(\text{uniform over } S)}(h)$.

The Goal

We wish to find some hypothesis, $h : \mathcal{X} \to \mathcal{Y}$, that (probably approximately) minimizes the true risk, $L_D(h)$.

The Bayes Optimal Predictor.
Given any probability distribution $\mathcal{D}$ over $\mathcal{X} \times \{0,1\}$, the best label predicting function from $\mathcal{X}$ to $\{0,1\}$ will be

$$f_{\mathcal{D}}(x) = \begin{cases} 1 & \text{if } \mathbb{P}[y=1|x] \geq 1/2 \\ 0 & \text{otherwise} \end{cases}$$

It is easy to verify (see Exercise 3.7) that for every probability distribution $\mathcal{D}$, the Bayes optimal predictor $f_{\mathcal{D}}$ is optimal, in the sense that no other classifier, $g : \mathcal{X} \to \{0,1\}$, has a lower error. That is, for every classifier g, $L_{\mathcal{D}}(f_{\mathcal{D}}) \leq L_{\mathcal{D}}(g)$.

Unfortunately, since we do not know $\mathcal{D}$, we cannot utilize this optimal predictor $f_{\mathcal{D}}$. What the learner does have access to is the training sample. We can now present the formal definition of agnostic PAC learnability, which is a natural extension of the definition of PAC learnability to the more realistic, nonrealizable, learning setup we have just discussed.

Clearly, we cannot hope that the learning algorithm will find a hypothesis whose error is smaller than the minimal possible error, that of the Bayes predictor. Furthermore, as we shall prove later, once we make no prior assumptions about the data-generating distribution, no algorithm can be guaranteed to find a predictor that is as good as the Bayes optimal one. Instead, we require that the learning algorithm will find a predictor whose error is not much larger than the best possible error of a predictor in some given benchmark hypothesis class. Of course, the strength of such a requirement depends on the choice of that hypothesis class.

Definition 3.3 (Agnostic PAC Learnability). A hypothesis class $\mathcal{H}$ is agnostic PAC learnable if there exist a function $m_{\mathcal{H}} : (0,1)^2 \to \mathbb{N}$ and a learning algorithm with the following property: For every $\epsilon, \delta \in (0,1)$ and for every distribution $\mathcal{D}$ over $\mathcal{X} \times \mathcal{Y}$, when running the learning algorithm on $m \geq m_{\mathcal{H}}(\epsilon, \delta)$ i.i.d. examples generated by $\mathcal{D}$, the algorithm returns a hypothesis h such that, with probability of at least $1 - \delta$ (over the choice of the m training examples),

$$L_{\mathcal{D}}(h) \leq \min_{h' \in \mathcal{H}} L_{\mathcal{D}}(h') + \epsilon.$$

Clearly, if the realizability assumption holds, agnostic PAC learning provides the same guarantee as PAC learning. In that sense, agnostic PAC learning generalizes the definition of PAC learning. When the realizability assumption does not hold, no learner can guarantee an arbitrarily small error. Nevertheless, under the definition of agnostic PAC learning, a learner can still declare success if its error is not much larger than the best error achievable by a predictor from the class $\mathcal{H}$. This is in contrast to PAC learning, in which the learner is required to achieve a small error in absolute terms and not relative to the best error achievable by the hypothesis class.

3.2.2 The Scope of Learning Problems Modeled

We next extend our model so that it can be applied to a wide variety of learning tasks. Let us consider some examples of different learning tasks.

- **Multiclass Classification** Our classification does not have to be binary. Take, for example, the task of document classification: We wish to design a program that

will be able to classify given documents according to topics (e.g., news, sports, biology, medicine). A learning algorithm for such a task will have access to examples of correctly classified documents and, on the basis of these examples, should output a program that can take as input a new document and output a topic classification for that document. Here, the *domain set* is the set of all potential documents. Once again, we would usually represent documents by a set of *features* that could include counts of different key words in the document, as well as other possibly relevant features like the size of the document or its origin. The *label set* in this task will be the set of possible document topics (so $\mathcal{Y}$ will be some large finite set). Once we determine our domain and label sets, the other components of our framework look exactly the same as in the papaya tasting example; Our *training sample* will be a finite sequence of (feature vector, label) pairs, the learner's output will be a function from the domain set to the label set, and, finally, for our measure of success, we can use the probability, over (document, topic) pairs, of the event that our predictor suggests a wrong label.

- **Regression** In this task, one wishes to find some simple *pattern* in the data – a functional relationship between the $\mathcal{X}$ and $\mathcal{Y}$ components of the data. For example, one wishes to find a linear function that best predicts a baby's birth weight on the basis of ultrasound measures of his head circumference, abdominal circumference, and femur length. Here, our domain set $\mathcal{X}$ is some subset of $\mathbb{R}^3$ (the three ultrasound measurements), and the set of "labels," $\mathcal{Y}$, is the the set of real numbers (the weight in grams). In this context, it is more adequate to call $\mathcal{Y}$ the *target* set. Our training data as well as the learner's output are as before (a finite sequence of (x, y) pairs, and a function from $\mathcal{X}$ to $\mathcal{Y}$ respectively). However, our measure of success is different. We may evaluate the quality of a hypothesis function, $h : \mathcal{X} \to \mathcal{Y}$, by the *expected square difference* between the true labels and their predicted values, namely,

$$L_{\mathcal{D}}(h) \stackrel{\text{def}}{=} \mathop{\mathbb{E}}_{(x,y)\sim\mathcal{D}} (h(x) - y)^2. \tag{3.2}$$

To accommodate a wide range of learning tasks we generalize our formalism of the measure of success as follows:

Generalized Loss Functions

Given any set $\mathcal{H}$ (that plays the role of our hypotheses, or models) and some domain Z let ℓ be any function from $\mathcal{H} \times Z$ to the set of nonnegative real numbers, $\ell : \mathcal{H} \times Z \to \mathbb{R}_+$. We call such functions *loss functions*.

Note that for prediction problems, we have that $Z = \mathcal{X} \times \mathcal{Y}$. However, our notion of the loss function is generalized beyond prediction tasks, and therefore it allows Z to be any domain of examples (for instance, in unsupervised learning tasks such as the one described in Chapter 22, Z is not a product of an instance domain and a label domain).

We now define the *risk function* to be the expected loss of a classifier, $h \in \mathcal{H}$, with respect to a probability distribution D over Z, namely,

$$L_{\mathcal{D}}(h) \stackrel{\text{def}}{=} \mathop{\mathbb{E}}_{z\sim\mathcal{D}} [\ell(h, z)]. \tag{3.3}$$

That is, we consider the expectation of the loss of h over objects z picked randomly according to $\mathcal{D}$. Similarly, we define the *empirical risk* to be the expected loss over a given sample $S = (z_1, \ldots, z_m) \in Z^m$, namely,

$$L_S(h) \stackrel{\text{def}}{=} \frac{1}{m} \sum_{i=1}^{m} \ell(h, z_i). \tag{3.4}$$

The loss functions used in the preceding examples of classification and regression tasks are as follows:

- **0–1 loss:** Here, our random variable z ranges over the set of pairs $\mathcal{X} \times \mathcal{Y}$ and the loss function is
$$\ell_{0-1}(h, (x, y)) \stackrel{\text{def}}{=} \begin{cases} 0 & \text{if } h(x) = y \\ 1 & \text{if } h(x) \neq y \end{cases}$$
This loss function is used in binary or multiclass classification problems.
One should note that, for a random variable, α, taking the values $\{0, 1\}$, $\mathbb{E}_{\alpha \sim D}[\alpha] = \mathbb{P}_{\alpha \sim D}[\alpha = 1]$. Consequently, for this loss function, the definitions of $L_{\mathcal{D}}(h)$ given in Equation (3.3) and Equation (3.1) coincide.
- **Square Loss:** Here, our random variable z ranges over the set of pairs $\mathcal{X} \times \mathcal{Y}$ and the loss function is
$$\ell_{\text{sq}}(h, (x, y)) \stackrel{\text{def}}{=} (h(x) - y)^2.$$
This loss function is used in regression problems.

We will later see more examples of useful instantiations of loss functions.

To summarize, we formally define agnostic PAC learnability for general loss functions.

Definition 3.4 (Agnostic PAC Learnability for General Loss Functions). A hypothesis class $\mathcal{H}$ is agnostic PAC learnable with respect to a set Z and a loss function $\ell : \mathcal{H} \times Z \to \mathbb{R}_+$, if there exist a function $m_{\mathcal{H}} : (0, 1)^2 \to \mathbb{N}$ and a learning algorithm with the following property: For every $\epsilon, \delta \in (0, 1)$ and for every distribution $\mathcal{D}$ over Z, when running the learning algorithm on $m \geq m_{\mathcal{H}}(\epsilon, \delta)$ i.i.d. examples generated by $\mathcal{D}$, the algorithm returns $h \in \mathcal{H}$ such that, with probability of at least $1 - \delta$ (over the choice of the m training examples),

$$L_{\mathcal{D}}(h) \leq \min_{h' \in \mathcal{H}} L_{\mathcal{D}}(h') + \epsilon,$$

where $L_{\mathcal{D}}(h) = \mathbb{E}_{z \sim \mathcal{D}}[\ell(h, z)]$.

Remark 3.1 (A Note About Measurability*). In the aforementioned definition, for every $h \in \mathcal{H}$, we view the function $\ell(h, \cdot) : Z \to \mathbb{R}_+$ as a random variable and define $L_{\mathcal{D}}(h)$ to be the expected value of this random variable. For that, we need to require that the function $\ell(h, \cdot)$ is measurable. Formally, we assume that there is a σ-algebra of subsets of Z, over which the probability $\mathcal{D}$ is defined, and that the preimage of every initial segment in $\mathbb{R}_+$ is in this σ-algebra. In the specific case of binary classification with the 0–1 loss, the σ-algebra is over $\mathcal{X} \times \{0, 1\}$ and our assumption on ℓ is equivalent to the assumption that for every h, the set $\{(x, h(x)) : x \in \mathcal{X}\}$ is in the σ-algebra.

Chapter 1
Introduction

We start with a brief overview of empirical risk minimization problems and of the role of empirical and Rademacher processes in constructing distribution dependent and data dependent excess risk bounds. We then discuss penalized empirical risk minimization and oracle inequalities and conclude with sparse recovery and low rank matrix recovery problems. Many important aspects of empirical risk minimization are beyond the scope of these notes, in particular, the circle of questions related to approximation theory (see well known papers by Cucker and Smale [47], DeVore et al. [49] and references therein).

1.1 Abstract Empirical Risk Minimization

Let $X, X_1, \dots, X_n, \dots$ be i.i.d. random variables defined on a probability space $(\Omega, \Sigma, \mathbb{P})$ and taking values in a measurable space $(S, \mathscr{A})$ with common distribution P. Let P_n denote the empirical measure based on the sample $(X_1, \dots, X_n)$ of the first n observations:

$$P_n := n^{-1} \sum_{j=1}^{n} \delta_{X_j},$$

where $\delta_x,\ x \in S$ is the Diracs's measure. Let $\mathscr{F}$ be a class of measurable functions $f : S \mapsto \mathbb{R}$. In what follows, the values of a function $f \in \mathscr{F}$ will be interpreted as "losses" associated with certain "actions" and the expectation of $f(X)$,

$$\mathbb{E} f(X) = \int_S f dP = Pf,$$

will be viewed as the risk of a certain "decision rule". We will be interested in the problem of risk minimization

$$Pf \longrightarrow \min,\ f \in \mathscr{F} \tag{1.1}$$

V. Koltchinskii, *Oracle Inequalities in Empirical Risk Minimization and Sparse Recovery Problems*, Lecture Notes in Mathematics 2033, DOI 10.1007/978-3-642-22147-7_1,

© Springer-Verlag Berlin Heidelberg 2011

in the cases when the distribution P is unknown and has to be estimated based on the data $(X_1, \dots, X_n)$. Since the empirical measure P_n is a natural estimator of P, the true risk can be estimated by the corresponding empirical risk,

$$n^{-1} \sum_{j=1}^{n} f(X_j) = \int_S f dP_n = P_n f,$$

and the risk minimization problem has to be replaced by the empirical risk minimization:

$$P_n f \longrightarrow \min, \ f \in \mathscr{F}. \tag{1.2}$$

Many important methods of statistical estimation such as maximum likelihood and more general M-estimation are versions of empirical risk minimization. The general theory of empirical risk minimization has started with seminal paper of Vapnik and Chervonenkis [147] (see Vapnik [146] for more references) although some important ideas go back to much earlier work on asymptotic theory of M-estimation. Vapnik and Chervonenkis were motivated by applications of empirical risk minimization in pattern recognition and learning theory that required the development of the theory in a much more general framework than what was common in statistical literature. Their key idea was to relate the quality of the solution of empirical risk minimization problem to the accuracy of approximation of the true distribution P by the empirical distribution P_n uniformly over function classes representing losses of decision rules. Because of this, they have studied general Glivenko–Cantelli problems about convergence of $\|P_n - P\|_{\mathscr{F}}$ to 0, where

$$\|Y\|_{\mathscr{F}} := \sup_{f \in \mathscr{F}} |Y(f)|$$

for $Y : \mathscr{F} \mapsto \mathbb{R}$. Vapnik and Chervonenkis introduced a number of important characteristics of complexity of function classes, such as VC-dimensions and random entropies, that control the accuracy of empirical approximation. These results along with the development of classical limit theorems in Banach spaces in the 1960s and 1970s led to the general theory of empirical processes that started with the pathbreaking paper by Dudley [58] on central limit theorems for empirical measures (see well known books by Dudley [59], Pollard [123], van der Vaart and Wellner [148]).

In the 1990s, Talagrand studied isoperimetric inequalities for product measures and, in particular, he proved a striking uniform version of Bernstein inequality describing concentration of $\|P_n - P\|_{\mathscr{F}}$ around its expectation (see Talagrand [138, 139]). This was a real breakthrough in the theory of empirical processes and empirical risk minimization. At about the same time, a concept of *oracle inequalities* has been developed in nonparametric statistics (see, e.g., Johnstone [74]). In modern statistics, it is common to deal with a multitude of possible models that describe the same data (for instance, a family of models for unknown regression functions of varying complexity). An oracle inequality is a bound on the risk of a statistical

estimator that shows that the performance of the estimator is almost (often, up to numerical constants) as good as it would be if the statistician had an access to an oracle that knows what the best model for the target function is. It happened that concentration inequalities provide rather natural probabilistic tools needed to develop oracle inequalities in a number of statistical problems. In particular, Birgé and Massart [23], Barron et al. [12], and, more recently, Massart [106, 107] suggested a general approach to model selection in a variety of statistical problems such as density estimation, regression and classification that is based on penalized empirical risk minimization. They used Talagrand's concentration and deviation inequalities in a systematic way to establish a number of oracle inequalities showing some form of optimality of penalized empirical risk minimization as a model selection tool.

In the recent years, new important classes of algorithms in machine learning have been introduced that are based on empirical risk minimization. In particular, large margin classification algorithms, such as boosting and support vector machines (SVM), can be viewed as empirical risk minimization over infinite dimensional functional spaces with special convex loss functions. In an attempt to understand the nature of these classification methods and to explain their superb generalization performance, there has been another round of work on the abstract theory of empirical risk minimization. One of the main ideas was to use the sup-norms or localized sup-norms of the Rademacher processes indexed by function classes to develop a general approach to measuring the complexities of these classes (see Koltchinskii [81], Bartlett et al. [14], Koltchinskii and Panchenko [92], Bousquet et al. [34], Bartlett et al. [15], Lugosi and Wegkamp [104], Bartlett and Mendelson [17]). This resulted in rather flexible definitions of distribution dependent and data dependent complexities in an abstract framework as well as more specialized complexities reflecting relevant parameters of specific learning machines. Moreover, such complexities have been used as natural penalties in model selection methods. This approach provided a general explanation of fast convergence rates in classification and other learning problems, the phenomenon discovered and studied by several authors, in particular, by Mammen and Tsybakov [105] and in an influential paper by Tsybakov [144].

1.2 Excess Risk: Distribution Dependent Bounds

Definition 1.1. Let

$$\mathscr{E}(f) := \mathscr{E}_P(f) := \mathscr{E}_P(\mathscr{F}; f) := Pf - \inf_{g \in \mathscr{F}} Pg.$$

This quantity will be called the *excess risk* of $f \in \mathscr{F}$.

Let

$$\hat{f} = \hat{f}_n \in \operatorname{Argmin}_{f \in \mathscr{F}} P_n f$$

4

Learning via Uniform Convergence

The first formal learning model that we have discussed was the PAC model. In Chapter 2 we have shown that under the realizability assumption, any finite hypothesis class is PAC learnable. In this chapter we will develop a general tool, *uniform convergence*, and apply it to show that any finite class is learnable in the agnostic PAC model with general loss functions, as long as the range loss function is bounded.

4.1 UNIFORM CONVERGENCE IS SUFFICIENT FOR LEARNABILITY

The idea behind the learning condition discussed in this chapter is very simple. Recall that, given a hypothesis class, $\mathcal{H}$, the ERM learning paradigm works as follows: Upon receiving a training sample, S, the learner evaluates the risk (or error) of each h in $\mathcal{H}$ on the given sample and outputs a member of $\mathcal{H}$ that minimizes this empirical risk. The hope is that an h that minimizes the empirical risk with respect to S is a risk minimizer (or has risk close to the minimum) with respect to the true data probability distribution as well. For that, it suffices to ensure that the empirical risks of all members of $\mathcal{H}$ are good approximations of their true risk. Put another way, we need that uniformly over all hypotheses in the hypothesis class, the empirical risk will be close to the true risk, as formalized in the following.

Definition 4.1 (ϵ-representative sample). A training set S is called ϵ-representative (w.r.t. domain Z, hypothesis class $\mathcal{H}$, loss function ℓ, and distribution $\mathcal{D}$) if

$$\forall h \in \mathcal{H}, \ \ |L_S(h) - L_{\mathcal{D}}(h)| \leq \epsilon.$$

The next simple lemma states that whenever the sample is $(\epsilon/2)$-representative, the ERM learning rule is guaranteed to return a good hypothesis.

Lemma 4.2. *Assume that a training set S is $\frac{\epsilon}{2}$-representative (w.r.t. domain Z, hypothesis class $\mathcal{H}$, loss function ℓ, and distribution $\mathcal{D}$). Then, any output of* $\mathrm{ERM}_{\mathcal{H}}(S)$*, namely, any $h_S \in \operatorname{argmin}_{h \in \mathcal{H}} L_S(h)$, satisfies*

$$L_{\mathcal{D}}(h_S) \ \leq \ \min_{h \in \mathcal{H}} L_{\mathcal{D}}(h) + \epsilon.$$

Proof. For every $h \in \mathcal{H}$,

$$L_{\mathcal{D}}(h_S) \leq L_S(h_S) + \frac{\epsilon}{2} \leq L_S(h) + \frac{\epsilon}{2} \leq L_{\mathcal{D}}(h) + \frac{\epsilon}{2} + \frac{\epsilon}{2} = L_{\mathcal{D}}(h) + \epsilon,$$

where the first and third inequalities are due to the assumption that S is $\frac{\epsilon}{2}$-representative (Definition 4.1) and the second inequality holds since h_S is an ERM predictor. □

The preceding lemma implies that to ensure that the ERM rule is an agnostic PAC learner, it suffices to show that with probability of at least $1 - \delta$ over the random choice of a training set, it will be an ϵ-representative training set. The uniform convergence condition formalizes this requirement.

Definition 4.3 (Uniform Convergence). We say that a hypothesis class $\mathcal{H}$ has the *uniform convergence property* (w.r.t. a domain Z and a loss function ℓ) if there exists a function $m_{\mathcal{H}}^{\mathrm{UC}} : (0,1)^2 \to \mathbb{N}$ such that for every $\epsilon, \delta \in (0,1)$ and for every probability distribution $\mathcal{D}$ over Z, if S is a sample of $m \geq m_{\mathcal{H}}^{\mathrm{UC}}(\epsilon, \delta)$ examples drawn i.i.d. according to $\mathcal{D}$, then, with probability of at least $1 - \delta$, S is ϵ-representative.

Similar to the definition of sample complexity for PAC learning, the function $m_{\mathcal{H}}^{\mathrm{UC}}$ measures the (minimal) sample complexity of obtaining the uniform convergence property, namely, how many examples we need to ensure that with probability of at least $1 - \delta$ the sample would be ϵ-representative.
The term *uniform* here refers to having a fixed sample size that works for all members of $\mathcal{H}$ and over all possible probability distributions over the domain.

The following corollary follows directly from Lemma 4.2 and the definition of uniform convergence.

Corollary 4.4. *If a class $\mathcal{H}$ has the uniform convergence property with a function $m_{\mathcal{H}}^{UC}$ then the class is agnostically PAC learnable with the sample complexity $m_{\mathcal{H}}(\epsilon, \delta) \leq m_{\mathcal{H}}^{UC}(\epsilon/2, \delta)$. Furthermore, in that case, the* $\mathrm{ERM}_{\mathcal{H}}$ *paradigm is a successful agnostic PAC learner for $\mathcal{H}$.*

4.2 FINITE CLASSES ARE AGNOSTIC PAC LEARNABLE

In view of Corollary 4.4, the claim that every finite hypothesis class is agnostic PAC learnable will follow once we establish that uniform convergence holds for a finite hypothesis class.

To show that uniform convergence holds we follow a two step argument, similar to the derivation in Chapter 2. The first step applies the union bound while the second step employs a measure concentration inequality. We now explain these two steps in detail.

Fix some ϵ, δ. We need to find a sample size m that guarantees that for any $\mathcal{D}$, with probability of at least $1 - \delta$ of the choice of $S = (z_1, \ldots, z_m)$ sampled i.i.d. from $\mathcal{D}$ we have that for all $h \in \mathcal{H}$, $|L_S(h) - L_{\mathcal{D}}(h)| \leq \epsilon$. That is,

$$\mathcal{D}^m(\{S : \forall h \in \mathcal{H}, |L_S(h) - L_{\mathcal{D}}(h)| \leq \epsilon\}) \geq 1 - \delta.$$

Equivalently, we need to show that

$$\mathcal{D}^m(\{S : \exists h \in \mathcal{H}, |L_S(h) - L_{\mathcal{D}}(h)| > \epsilon\}) < \delta.$$

Writing

$$\{S : \exists h \in \mathcal{H}, |L_S(h) - L_{\mathcal{D}}(h)| > \epsilon\} = \cup_{h \in \mathcal{H}} \{S : |L_S(h) - L_{\mathcal{D}}(h)| > \epsilon\},$$

and applying the union bound (Lemma 2.2) we obtain

$$\mathcal{D}^m(\{S : \exists h \in \mathcal{H}, |L_S(h) - L_{\mathcal{D}}(h)| > \epsilon\}) \leq \sum_{h \in \mathcal{H}} \mathcal{D}^m(\{S : |L_S(h) - L_{\mathcal{D}}(h)| > \epsilon\}). \quad (4.1)$$

Our second step will be to argue that each summand of the right-hand side of this inequality is small enough (for a sufficiently large m). That is, we will show that for any fixed hypothesis, h, (which is chosen in advance prior to the sampling of the training set), the gap between the true and empirical risks, $|L_S(h) - L_{\mathcal{D}}(h)|$, is likely to be small.

Recall that $L_{\mathcal{D}}(h) = \mathbb{E}_{z \sim \mathcal{D}}[\ell(h, z)]$ and that $L_S(h) = \frac{1}{m}\sum_{i=1}^m \ell(h, z_i)$. Since each z_i is sampled i.i.d. from $\mathcal{D}$, the expected value of the random variable $\ell(h, z_i)$ is $L_{\mathcal{D}}(h)$. By the linearity of expectation, it follows that $L_{\mathcal{D}}(h)$ is also the expected value of $L_S(h)$. Hence, the quantity $|L_{\mathcal{D}}(h) - L_S(h)|$ is the deviation of the random variable $L_S(h)$ from its expectation. We therefore need to show that the measure of $L_S(h)$ is *concentrated* around its expected value.

A basic statistical fact, the *law of large numbers*, states that when m goes to infinity, empirical averages converge to their true expectation. This is true for $L_S(h)$, since it is the empirical average of m i.i.d random variables. However, since the law of large numbers is only an asymptotic result, it provides no information about the gap between the empirically estimated error and its true value for any given, finite, sample size.

Instead, we will use a measure concentration inequality due to Hoeffding, which quantifies the gap between empirical averages and their expected value.

Lemma 4.5 (Hoeffding's Inequality)**.** *Let $\theta_1, \ldots, \theta_m$ be a sequence of i.i.d. random variables and assume that for all i, $\mathbb{E}[\theta_i] = \mu$ and $\mathbb{P}[a \leq \theta_i \leq b] = 1$. Then, for any $\epsilon > 0$*

$$\mathbb{P}\left[\left|\frac{1}{m}\sum_{i=1}^m \theta_i - \mu\right| > \epsilon\right] \leq 2\exp\left(-2m\epsilon^2/(b-a)^2\right).$$

The proof can be found in Appendix B.

Getting back to our problem, let θ_i be the random variable $\ell(h, z_i)$. Since h is fixed and $z_1, \ldots, z_m$ are sampled i.i.d., it follows that $\theta_1, \ldots, \theta_m$ are also i.i.d. random variables. Furthermore, $L_S(h) = \frac{1}{m}\sum_{i=1}^m \theta_i$ and $L_{\mathcal{D}}(h) = \mu$. Let us further assume that the range of ℓ is $[0, 1]$ and therefore $\theta_i \in [0, 1]$. We therefore obtain that

$$\mathcal{D}^m(\{S : |L_S(h) - L_{\mathcal{D}}(h)| > \epsilon\}) = \mathbb{P}\left[\left|\frac{1}{m}\sum_{i=1}^m \theta_i - \mu\right| > \epsilon\right] \leq 2\exp\left(-2m\epsilon^2\right). \quad (4.2)$$

Combining this with Equation (4.1) yields

$$\begin{aligned}\mathcal{D}^m(\{S : \exists h \in \mathcal{H}, |L_S(h) - L_{\mathcal{D}}(h)| > \epsilon\}) &\leq \sum_{h \in \mathcal{H}} 2\exp\left(-2m\epsilon^2\right) \\ &= 2|\mathcal{H}|\exp\left(-2m\epsilon^2\right).\end{aligned}$$

Finally, if we choose

$$m \geq \frac{\log(2|\mathcal{H}|/\delta)}{2\epsilon^2}$$

then

$$\mathcal{D}^m(\{S : \exists h \in \mathcal{H}, |L_S(h) - L_{\mathcal{D}}(h)| > \epsilon\}) \leq \delta.$$

Corollary 4.6. *Let $\mathcal{H}$ be a finite hypothesis class, let Z be a domain, and let $\ell : \mathcal{H} \times Z \to [0,1]$ be a loss function. Then, $\mathcal{H}$ enjoys the uniform convergence property with sample complexity*

$$m_{\mathcal{H}}^{UC}(\epsilon, \delta) \leq \left\lceil \frac{\log(2|\mathcal{H}|/\delta)}{2\epsilon^2} \right\rceil.$$

Furthermore, the class is agnostically PAC learnable using the ERM algorithm with sample complexity

$$m_{\mathcal{H}}(\epsilon, \delta) \leq m_{\mathcal{H}}^{UC}(\epsilon/2, \delta) \leq \left\lceil \frac{2\log(2|\mathcal{H}|/\delta)}{\epsilon^2} \right\rceil.$$

Remark 4.1 (The "Discretization Trick"). While the preceding corollary only applies to finite hypothesis classes, there is a simple trick that allows us to get a very good estimate of the practical sample complexity of infinite hypothesis classes. Consider a hypothesis class that is parameterized by d parameters. For example, let $\mathcal{X} = \mathbb{R}$, $\mathcal{Y} = \{\pm 1\}$, and the hypothesis class, $\mathcal{H}$, be all functions of the form $h_\theta(x) = \text{sign}(x - \theta)$. That is, each hypothesis is parameterized by one parameter, $\theta \in \mathbb{R}$, and the hypothesis outputs 1 for all instances larger than θ and outputs -1 for instances smaller than θ. This is a hypothesis class of an infinite size. However, if we are going to learn this hypothesis class in practice, using a computer, we will probably maintain real numbers using floating point representation, say, of 64 bits. It follows that in practice, our hypothesis class is parameterized by the set of scalars that can be represented using a 64 bits floating point number. There are at most 2^{64} such numbers; hence the actual size of our hypothesis class is at most 2^{64}. More generally, if our hypothesis class is parameterized by d numbers, in practice we learn a hypothesis class of size at most 2^{64d}. Applying Corollary 4.6 we obtain that the sample complexity of such classes is bounded by $\frac{128d + 2\log(2/\delta)}{\epsilon^2}$. This upper bound on the sample complexity has the deficiency of being dependent on the specific representation of real numbers used by our machine. In Chapter 6 we will introduce a rigorous way to analyze the sample complexity of infinite size hypothesis classes. Nevertheless, the discretization trick can be used to get a rough estimate of the sample complexity in many practical situations.

4.3 SUMMARY

If the uniform convergence property holds for a hypothesis class $\mathcal{H}$ then in most cases the empirical risks of hypotheses in $\mathcal{H}$ will faithfully represent their true risks. Uniform convergence suffices for agnostic PAC learnability using the ERM rule. We have shown that finite hypothesis classes enjoy the uniform convergence property and are hence agnostic PAC learnable.